NODEBASE · EDITOR INTERNALS

Visual Editor: Node Add, Client State & Save to PostgreSQL

End-to-end trace of the React Flow canvas: how a node is created in the browser, how its state is held across three separate stores, and how a click of **Save** becomes an upsert transaction against the `Node` and `Connection` tables.

SOURCE OF TRUTH checked-out working tree DB EVIDENCE live Neon PostgreSQL LIBRARY @xyflow/react 12.11.6

Scope and method

This document follows one node from the moment a user picks it from the side panel to the moment its row lands in the database. Every file reference was read directly in the working tree, and every database claim was produced by querying the live database (`information_schema` and `pg_constraint`) rather than inferred from the schema file alone.

The editor is worth documenting carefully because it does not have a single source of truth. Three independent stores hold the graph, and the save button reads from the one that React owns — not the two React owns itself. Getting that distinction right explains both the happy path and several of the defects at the end of this document.

CONCERN	PRIMARY FILE
Route + server prefetch	`src/app/(dashboard)/(editor)/workflows/[workflowId]/page.tsx`
Canvas + change handlers	`src/features/editor/components/editor.tsx`
Node creation UI	`src/components/node-selector.tsx`
Per-node config dialogs	`src/features/executions/components/*/node.tsx + dialog.tsx`
Save button	`src/features/editor/components/editor-header.tsx`
Persistence	`src/features/workflows/server/routers.ts`
Table definitions	`prisma/schema.prisma`
Execution ordering	`src/inngest/utils.ts, src/inngest/functions.ts`

PHASE 1 · ROUTE AND DATA LOAD

---

1 Authenticate, then prefetch

The page is an async server component. It awaits a real Better Auth session via `requireAuth()` and then calls `prefetchWorkflow(workflowId)` before rendering anything. Note that the tRPC context factory is a stub returning a hardcoded `userId: "user_123"` — authorization is *not* inherited from context. It is re-established inside every procedure.

`page.tsx:20-23`

2 Dehydrate the prefetch into the client cache

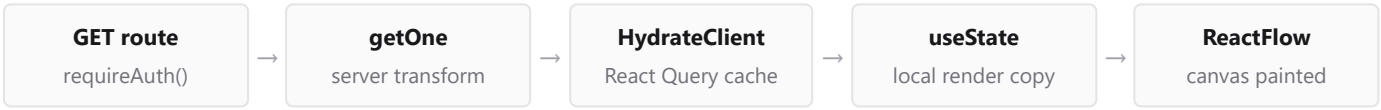
`prefetchWorkflow` issues `trpc.workflows.getOne.prefetch({ id })`, and `<HydrateClient>` serialises that result into the React Query cache. This is why `<Editor>` can call a *suspense* query and render on first paint with no client-side waterfall: the data is already in the cache when React hydrates.

The header and the canvas sit inside the same `ErrorBoundary` + `Suspense` pair, so a failed prefetch surfaces as `<EditorError />` rather than a blank route.

3 Canvas mounts from the hydrated query

`useSuspenseWorkflow(workflowId)` inside `<Editor>` reads that hydrated entry. The returned `nodes` and `edges` arrays are copied into component state, and that state is what React Flow renders.

`editor.tsx:43-44`



PHASE 2 · SERVER ROWS BECOME REACT FLOW ELEMENTS

The `getOne` query is the only place where the database's node/edge vocabulary is translated into React Flow's. Ownership is enforced inside the `where` clause rather than in a separate check, so a foreign `workflowId` is indistinguishable from a missing one — both surface as `findUniqueOrThrow`.

```
const workflow = await prisma.workflow.findUniqueOrThrow({
  where: { id: input.id, userId: ctx.auth.user.id },
  include: { nodes: true, connections: true },
});
```

`routers.ts:218-227`

The transformation is a flat rename, and it is worth writing out because the column names in PostgreSQL are deliberately *not* the names React Flow uses:

DATABASE COLUMN	REACT FLOW FIELD	NOTES
`Node.id`	`node.id`	Client-generated cuid2, so save is an upsert
`Node.type`	`node.type`	Postgres enum cast to string
`Node.position (jsonb)`	`node.position`	Cast to `{x:number,y:number}`
`Node.data (jsonb)`	`node.data`	`|| {}` guards a SQL NULL
`Connection.id`	`edge.id`	Reused as the edge key
`Connection.fromNodeId`	`edge.source`	—
`Connection.toNodeId`	`edge.target`	—
`Connection.fromOutput`	`edge.sourceHandle`	Defaults to `main` in the schema
`Connection.toInput`	`edge.targetHandle`	Defaults to `main` in the schema

`routers.ts:229-244`

> **Why handles matter**
> `fromOutput / toInput` are the bridge between a visual handle on a custom node and a row in `Connection`. Because React Flow auto-names unlabelled handles `source-1` and `target-1`, the schema's `"main"` defaults are almost never the values actually stored — confirmed against live data in Phase 9.

PHASE 3 · THREE STORES HOLD THE GRAPH

This is the single most important structural fact in the editor. The graph exists in three places at once, and they are kept in agreement by React Flow rather than by application code.

#	STORE	OWNER	WHO WRITES	WHO READS
1	`useState<Node[]> / useState<Edge[]>`	React	`applyNodeChanges`, `applyEdgeChanges`, `addEdge`	`<ReactFlow nodes edges>` (render)
2	`React Flow internal store`	`@xyflow/system`	`useReactFlow().setNodes`, user gestures	pan/zoom, minimap, hit-testing
3	`editorAtom (Jotai)`	App	`onInit={setEditor}`	**The Save button**

Store 3 is a `ReactFlowInstance` captured once on mount. The canvas hands it over through the `onInit` prop and the app parks it in a Jotai atom:

```
const setEditor = useSetAtom(editorAtom);

<ReactFlow
  nodes={nodes}
  edges={edges}
  onNodesChange={onNodesChange}
  onEdgesChange={onEdgesChange}
  onConnect={onConnect}
```

```
  nodeTypes={nodeComponents}
  onInit={setEditor}
  fitView
  snapGrid={[10, 10]}
  snapToGrid
  panOnScroll
  panOnDrag={false}
  selectionOnDrag
>
```

`editor.tsx:40, 82-96`

> **The Save button never reads the React state**
> It calls `editor.getNodes()` and `editor.getEdges()` on the instance from the atom — so what is persisted is React Flow's internal store, not the `useState` arrays that this same file declares. Correctness therefore depends entirely on React Flow propagating store changes back into the controlled props. That propagation is not a coincidence; it is the queue mechanism verified in Phase 6.

The second atom, `isDirtyAtom` (default `false`), is the single boolean that gates the Save button. It lives in Jotai rather than component state so that the header — a different component from the canvas — can read and clear it.

PHASE 4 · ADDING A NODE

1 Open the picker

`<AddNodeButton />` is rendered in a `Panel` at the top-right of the canvas. It opens `NodeSelector`, a right-hand `Sheet` that lists the full node catalogue.

2 Guard the single-instance triggers

Before anything is created, the handler reads the *current* nodes via `getNodes()` and enforces two client-side uniqueness rules, each rejecting with a toast and an early `return`:

`MANUAL_TRIGGER` — one manual trigger per workflow. `CRON_TRIGGER` — one schedule per workflow, because each one owns a BullMQ repeatable job.

These checks are advisory only. Nothing on the server re-validates them, so a hand-rolled API call can still create a second trigger.

3 Construct the node

The new node is built inside a functional update to `setNodes`, so the latest array is used:

```
const flowPosition = screenToFlowPosition({
  x: centerX + (Math.random() - 0.5) * 200,
  y: centerY + (Math.random() - 0.5) * 200,
});

const newNode = {
  id: createId(),
  data: {},
  position: flowPosition,
  type: selection.type,
};
```

`node-selector.tsx:163-176`

Four deliberate choices here: the id is generated **client-side** with cuid2, so the later write is an upsert keyed on a value the browser already holds; `data` starts empty because configuration is the dialog's job; the position is derived from the *viewport centre* plus a ±100 px jitter, converted out of screen space; and only `type` is set — no `name`, no `width`, no `selected`.

4 The node is appended

```
if (hasInitialTrigger) return [newNode];

return [...nodes, newNode];
```

`node-selector.tsx:178-180`

> **Defect — this branch discards the entire canvas**
>
> When the current graph contains a node of type `INITIAL`, the callback returns an array containing *only* the new node. Every pre-existing node is dropped. The intent was evidently to *replace* an implicit starter node, but the guard is written as a whole-array return rather than a filter, so the blast radius is the entire workflow. Ten live workflows still contain an `INITIAL` node (Phase 9), so this path is reachable today. It is analysed in full in Finding 1.

PHASE 5 · CONFIGURING A NODE

Configuration is not a separate server round-trip. Each node component owns its own dialog and writes straight into React Flow's store, exactly like the selector does.

Using `HttpRequestNode` as the reference implementation:

1 Local dialog visibility

The node component is a `memo`-wrapped function holding `const [dialogOpen, setDialogOpen] = useState(false)`. Both a gear affordance and a double-click open it, via `onSettings` and `onDoubleClick` passed to the shared `BaseExecutionNode` chrome.

`http-request/node.tsx:23, 35, 66-72`

2 Form seeded from persisted data

`defaultValues={nodeData}` hands the node's existing `data` to a `react-hook-form` instance validated by `zodResolver`. The schema is a discriminated set of the fields that node type needs — for HTTP: `variableName` (regex-validated identifier), `endpoint`, `method`, `body`. Validation is per-node-type, and a node type that has no dialog simply keeps `data: {}`.

3 Merge into `node.data`

```
const handleSubmit = (values: HttpRequestFormValues) => {
  setNodes((nodes) =>
    nodes.map((node) => {
      if (node.id === props.id) {
        return { ...node, data: { ...node.data, ...values } };
      }
      return node;
    }),
  );
};
```

`http-request/node.tsx:37-52`

The `data` object is spread-merged rather than replaced, so fields belonging to a different concern are preserved. The node card re-renders from `props.data` immediately: `nodeData?.endpoint` decides whether the card reads `GET: https://…` or the placeholder `Not configured`. Nothing is sent to the server until Save.

> **Where configuration physically lives**
>
> The dialog's output becomes `Node.data`, a **jsonb** column with no schema of its own. There is no per-node-type column, no validation at the database layer, and no partial-update endpoint — `data` is written wholesale whenever Save runs. This is why the server treats an empty object as "not loaded" rather than "clear it" (Phase 7).

Separately, each node subscribes to realtime execution status through `useNodeStatus({ nodeId, channel, topic: "status", refreshToken })`, which fetches a short-lived token from a colocated server action. The published shape on every channel is `{ nodeId, status: "loading" | "success" | "error" }`. This is a read path and has nothing to do with persistence.

PHASE 6 · HOW CHANGES PROPAGATE (VERIFIED IN LIBRARY SOURCE)

Because two of the three write paths (`setNodes` from the selector and from the dialogs) do not call `setDirty` themselves, the dirty flag depends on React Flow. The mechanism was read out of the installed package rather than assumed.

`useReactFlow().setNodes` does **not** write directly. It enqueues the update, and `BatchProvider` drains the queue: it folds every queued payload over the current array, then diffs the result against the internal lookup table.

```
let next = nodes;
for (const payload of queueItems) {
  next = typeof payload === 'function' ? payload(next) : payload;
}
let changes = getElementsDiffChanges({ items: next, lookup: nodeLookup });
...
// We only want to fire onNodesChange if there are changes to the nodes
if (changes.length > 0) {
  onNodesChange?.(changes);
}
```

`@xyflow/react 12.11.6 → dist/esm/index.js (BatchProvider)`

The diff emits exactly three change kinds:

CHANGE	CONDITION	TRIGGERS
`add`	id present in the new array, absent from the lookup	Node-selector drop, `addNodes`
`replace`	id exists but the object reference differs	Dialog config write, position drag
`remove`	id in the lookup, absent from the new array	Delete key, cut

The canvas handlers then translate those changes into the dirty flag:

```
const onNodesChange = useCallback((changes: NodeChange[]) => {
  if (changes.some(
    (change) => change.type !== "select" && change.type !== "dimensions",
  )) {
    setDirty(true);
  }
  setNodes((nodesSnapshot) => applyNodeChanges(changes, nodesSnapshot));
}, [setDirty]);

const onEdgesChange = useCallback((changes: EdgeChange[]) => {
  if (changes.some((change) => change.type !== "select")) {
    setDirty(true);
  }
  setEdges((edgesSnapshot) => applyEdgeChanges(changes, edgesSnapshot));
}, [setDirty]);

const onConnect = useCallback((params: Connection) => {
  setDirty(true);
  setEdges((edgesSnapshot) => addEdge(params, edgesSnapshot));
}, [setDirty]);
```

`editor.tsx:46-74`

The dirty logic is correct, and slightly elegant

Because the diff is content-aware, the two exclusions are both justified rather than defensive: `dimensions` is dropped because React Flow measures every node on mount and would otherwise mark a freshly loaded workflow dirty immediately, and `select` is dropped because clicking a node changes no persisted state. A user who opens a dialog and submits *identical* values produces no diff, so the Save button correctly stays disabled.

The button reflects the flag visually and functionally — an amber "Unsaved changes" dot while dirty, and `disabled={!isDirty || saveWorkflow.isPending}` so a double-click cannot fire two concurrent transactions.

PHASE 7 · THE SAVE CLICK

1 Snapshot the graph

```
const nodes = editor.getNodes();
const edges = editor.getEdges();

const input = { id: workflowId, nodes, edges };
```

`editor-header.tsx:38-49`

There is no debounce and no autosave — the graph is snapshotted on demand. `getNodes()` returns fully-hydrated React Flow node objects carrying `measured`, `selected`, `width` and `height`; all of that is stripped on arrival by the Zod schema in Phase 8, which only declares `id`, `type`, `position` and `data`. Zod's default object behaviour silently discards unknown keys, so the inflated payload is harmless — merely larger than it needs to be.

2 Mutate, then clear the flag on success only

`useUpdateWorkflow()` wraps the tRPC mutation with a toast on success and an error toast on failure, then invalidates both `workflows.getMany` (workflow list name/updatedAt) and `workflows.getOne` (fresh graph). The component clears the flag only after `mutateAsync` resolves; the `catch` block is intentionally empty because the mutation already toasted. A failed save therefore leaves the canvas dirty and the Save button re-enabled, which is the correct recovery behaviour.

3 Reconcile cron jobs

After the graph is committed, `syncCronSchedules(workflowId)` runs as a server action to reconcile BullMQ repeatable jobs in Redis so they match the saved cron triggers. This couples a node-type concern to a save — adding or removing a `CRON_TRIGGER` is not only a database change. The mechanism is documented separately in *BullMQ Worker & Queue Flow*.

`editor-header.tsx:56-61`

PHASE 8 · THE PERSISTENCE TRANSACTION

Input contract

```
z.object({
  id: z.string(),
  nodes: z.array(z.object({
    id: z.string(),
    type: z.string().nullish(),
    position: z.object({ x: z.number(), y: z.number() }),
    data: z.record(z.string(), z.any()).optional(),
  })),
  edges: z.array(z.object({
    source: z.string(),
    target: z.string(),
    sourceHandle: z.string().nullish(),
    targetHandle: z.string().nullish(),
  })),
})
```

`routers.ts:101-121`

Two observations. `type` is a free *string*, not the enum, so validation is deferred to the Prisma cast — an unknown type produces a database error, not a tidy input error. And the edge schema omits `id`, which is what makes the full edge replacement on line 177 safe.

Ownership

A pre-flight `findUniqueOrThrow` scoped by `{ id, userId: ctx.auth.user.id }` runs *before* the transaction. It serves two purposes: it authorises the write, and its `include: { nodes: true }` result supplies `existingNodeIds`, the baseline for computing deletions. The scoping is a genuine defence — the later `tx.node.deleteMany` and the node upsert keys are *not* re-scoped by user, so this check is load-bearing rather than decorative.

Nodes — upsert and merge, never wipe

```
for (const node of nodes) {
  const updateData: Prisma.NodeUpdateWithoutWorkflowInput = {
    position: node.position,
  };

  if (node.type) {
    updateData.type = node.type as NodeType;
    updateData.name = node.type;
  }

  //Only overwrite data when the client actually sent data. An empty
  //object means the config wasn't loaded and we should keep what is in
  //the database.
  if (node.data && Object.keys(node.data).length > 0) {
    updateData.data = node.data;
  }

  await tx.node.upsert({
    where: { id: node.id },
    create: {
      id: node.id,
      workflowId: id,
      name: node.type || "unknown",
      type: (node.type || NodeType.INITIAL) as NodeType,
```

```
      position: node.position,
      data: node.data || {},
    },
    update: updateData,
  });
}
```

`routers.ts:135-164`

The in-code comment states the design intent plainly: nodes are **merged**, not deleted-and-recreated, so a partially-loaded snapshot cannot wipe existing configuration. Three subtleties are worth calling out:

- `name` is not user-editable. It is derived from `type` on both create and update, so the column is effectively a denormalised copy of the type string.
- The empty-object guard means **config can never be cleared through this endpoint**. Sending `data: {}` is interpreted as "I have no config" and is silently ignored — there is no way to reset a node back to unconfigured.
- The create-path fallback `node.type || NodeType.INITIAL` silently persists an `INITIAL` node whenever the client omits `type`, because `type` is declared `nullish`. This is the most likely origin of the legacy rows described in Phase 9.

Removals

```
const removedNodeIds = [...existingNodeIds].filter(
  (nodeId) => !nodes.some((node) => node.id === nodeId),
);
if (removedNodeIds.length > 0) {
  await tx.node.deleteMany({
    where: { id: { in: removedNodeIds }, workflowId: id },
  });
}
```

`routers.ts:166-174`

Deletions are inferred by set difference against the pre-flight snapshot — the client never sends "delete this id". The `ON DELETE CASCADE` foreign keys on `Connection.fromNodeId` and `toNodeId` mean the attached edges disappear with the node, so no separate edge cleanup is needed for removals.

Edges — full replacement

```
//Replace connections
await tx.connection.deleteMany({ where: { workflowId: id } });

if (edges.length > 0) {
  await tx.connection.createMany({
    data: edges.map((edge) => ({
      workflowId: id,
      fromNodeId: edge.source,
      toNodeId: edge.target,
      fromOutput: edge.sourceHandle || "main",
      toInput: edge.targetHandle || "main",
    })),
  });
```

```
}

await tx.workflow.update({ where: { id }, data: { updatedAt: new Date() } });
```

`routers.ts:176-197`

Edges take the opposite approach to nodes: every save wipes and rebuilds the whole set. It is defensible — the diff would be more code for the same result, and the client already sends the complete graph — but it has a side effect worth noting. Because `Connection.id` is `@default(cuid())` and the input schema has no `id` field, **every connection gets a new primary key on every save**, along with fresh `createdAt` / `updatedAt` values. Edge identity is not stable across saves, so nothing may key off it.

> **Return value**
> The transaction returns the `workflow` object captured by the pre-flight read, i.e. a *pre-mutation* snapshot whose `nodes` are stale. It is benign today — the hook only reads `data.name` and `data.id` for its toast and invalidation keys, and `getOne` is refetched anyway — but it is a trap for any future caller that trusts the response.

`PHASE 9 · DATABASE TABLES (VERIFIED AGAINST LIVE NEON)`

Column names, types and constraints below were read from `information_schema.columns` and `pg_constraint`, not copied from the schema file. Only the three Better Auth models carry `@@map` directives; every application table is created with its Prisma model name, so these are genuinely mixed-case, quoted identifiers in PostgreSQL.

"Workflow"

COLUMN	TYPE	ROLE
`id`	text	cuid primary key
`userId`	text	Owner → `user(id)`, **ON DELETE CASCADE**
`name`	text	Edited by `EditorNameInput` via `updateName`
`createdAt`	timestamp	Default now
`updatedAt`	timestamp	Bumped explicitly at the end of every save

Physical table `"Workflow"` · constraints: `Workflow_pkey`, `Workflow_userId_fkey`

"Node"

COLUMN	SQL TYPE	ROLE
`id`	text	cuid2 generated in the browser, so the write is an upsert
`workflowId`	text	Parent → `"Workflow"(id)`, **ON DELETE CASCADE**
`name`	text	Denormalised copy of `type`; not user-editable
`type`	USER-DEFINED	`NodeType` enum — 15 variants
`position`	**jsonb**	`{"x":…, "y":…}`; canvas coordinates
`data`	**jsonb**	Per-type config from the dialog; shape unconstrained
`createdAt`	timestamp	Default now
`updatedAt`	timestamp	Prisma `@updatedAt`
`credentialId`	text	→ `"Credential"(id)`, **ON DELETE SET NULL** — *nullable and unused*

Physical table `"Node"` · constraints: `Node_pkey`, `Node_workflowId_fkey`, `Node_credentialId_fkey`

"Connection"

COLUMN	SQL TYPE	ROLE
`id`	text	cuid primary key — regenerated on every save
`workflowId`	text	Parent → `"Workflow"(id)`, **ON DELETE CASCADE**
`fromNodeId`	text	→ `"Node"(id)`, **ON DELETE CASCADE** (relation "FromNode")
`toNodeId`	text	→ `"Node"(id)`, **ON DELETE CASCADE** (relation "ToNode")
`fromOutput`	text	Default `main`; receives `edge.sourceHandle`
`toInput`	text	Default `main`; receives `edge.targetHandle`
`createdAt`	timestamp	Default now
`updatedAt`	timestamp	Prisma `@updatedAt`

Physical table `"Connection"` · constraints: `Connection_pkey`, `Connection_workflowId_fkey`, `Connection_fromNodeId_fkey`, `Connection_toNodeId_fkey`, plus the composite unique `@@unique([fromNodeId, toNodeId, fromOutput, toInput])`

The composite unique index is the only thing preventing duplicate parallel edges between the same pair of handles. Because the edge input schema has no `id` and the whole set is rebuilt inside one transaction, a duplicate submitted twice would surface as a unique-violation rather than silently double-inserting.

Live data snapshot

```
// aggregate counts, live database
workflows   34
nodes       46
connections 22
executions  66  across 11 workflows

// referential integrity, Connection -> Node
fromNodeId missing  0
```

```
toNodeId  missing     0
workflowId missing     0
orphan nodes          0

// handle values actually stored
source-1 -> target-1 : 22   of 22
  // the schema default "main" is never used

// node count per workflow, sorted
0 x9  1 x11  2 x11  3 x3  6 x1
```

Node types present in the live database:

NODETYPE	COUNT	TYPICAL DATA
MANUAL_TRIGGER	15	{}
HTTP_REQUEST	13	{ "body": "", "method": "GET", "endpoint": "https://reqres.in/api/users/2", "variableName": "gopi" }
INITIAL	10	{}
GEMINI	4	model / prompt config
TELEGRAM_SEND_MESSAGE	2	chat + text
TELEGRAM_SEND_BUTTONS	1	inline keyboard
GOOGLE_FORM_TRIGGER	1	form id

Every `INITIAL` row is a legacy artifact: all ten sit at `position {"x":0,"y":0}` with `data {}`, and each is the *only* node in its workflow. Positions for genuinely user-created nodes are real canvas coordinates in the 470–770 px band. The current `node-selector.tsx` never creates an `INITIAL` node, and `workflows.create` inserts no starter node, so these rows must predate the current code — most plausibly written by the `node.type || NodeType.INITIAL` fallback on the create path. They are inert until a user opens the workflow and adds a node, at which point Finding 1 triggers.

Real edges confirm the trigger-to-action shape the executor expects:

```
// sampled from live Connection rows, joined to Node
MANUAL_TRIGGER  -> HTTP_REQUEST
MANUAL_TRIGGER  -> GEMINI
MANUAL_TRIGGER  -> HTTP_REQUEST
HTTP_REQUEST     -> HTTP_REQUEST
GEMINI            -> TELEGRAM_SEND_MESSAGE
GEMINI            -> TELEGRAM_SEND_BUTTONS
```

PHASE 10 · HANDOFF TO EXECUTION

Saving a graph does not run it. The Inngest function `executeWorkflow` later reads the same two tables and imposes an execution order with `topologicalSort`.

```
const edges: [string, string][] = connections.map((conn) => [
  conn.fromNodeId,
  conn.toNodeId,
]);

// Isolated nodes get a self-loop so toposort does not discard them.
for (const node of nodes) {
  if (!connectedNodeIds.has(node.id)) {
    edges.push([node.id, node.id]);
  }
}

try {
  sortedNodeIds = toposort(edges);
  sortedNodeIds = [...new Set(sortedNodeIds)];
} catch (error) {
  if (error instanceof Error && error.message.includes("Cyclic")) {
    throw new Error("Workflow contains a cycle");
  }
  throw error;
}
```

`src/inngest/utils.ts`

Two design points carry over from the editor. The self-loop trick means a node with no edges is still executed rather than being silently dropped, and the `Set` de-duplication absorbs the doubled entries that self-loops produce. But the cycle check runs **only at execution time** — the editor places no constraint against connecting a node back into its own ancestry, and `workflows.update` does not validate acyclicity. A user can save a cyclic workflow successfully and only meet "Workflow contains a cycle" when they press Execute. The live database happens to contain no cycles (verified by depth-first search over all 34 workflows), so the condition is latent rather than active.



Findings

1 · CRITICAL — adding a node to a legacy workflow destroys the canvas

`node-selector.tsx:178`

`if (hasInitialTrigger) return [newNode];` returns an array holding only the new node, so every pre-existing node is discarded. Per the diff semantics in Phase 6 this emits one `add` plus N `remove` changes, the canvas collapses to a single node, and pressing Save deletes the rest of the workflow from `"Node"` (cascading their connections). **Ten live workflows still contain an `INITIAL` node**, so this is reachable today. If the intent was to swap an implicit starter node, the correct form is `nodes.filter(n => n.type !== NodeType.INITIAL).concat(newNode)` . Given that the selector never creates `INITIAL` nodes, deleting the branch entirely is likely the better fix.

2 — node configuration cannot be cleared

The `Object.keys(node.data).length > 0` guard at `routers.ts:148` is a sound protection against wiping config on a partial snapshot, but it is unconditional and there is no alternative endpoint. A user who wants to reset a configured node to blank has no path through the UI. A distinct `data === undefined` (absent) versus `{}` (explicit clear) distinction would resolve this, since the schema already types `data` as optional.

3 — connection identity churns on every save

Edges are deleted and recreated with fresh cuids (`routers.ts:177-191`), so `Connection.id` and `createdAt` change on every Save even when the graph is byte-identical. Harmless today because nothing references a connection by id, but it rules out caching, auditing, or last-modified semantics on edges.

4 — cycles are only caught at execution

React Flow permits a connection back into an ancestor and the save path performs no acyclicity check, so an invalid workflow persists cleanly and fails later in Inngest with "Workflow contains a cycle". A topological validation inside `workflows.update` would surface the error at the moment the user can still act on it.

5 — `INITIAL` is a silent fallback for a missing type

Because the input schema allows `type: nullish` and the create path defaults to `NodeType.INITIAL` (`routers.ts:158`), any client that omits a type persists an `INITIAL` node rather than failing validation. This almost certainly produced the ten orphaned rows now sitting in the database, and it is what makes Finding 1 reachable. Narrowing the field to the `NodeType` enum would convert a silent bad write into a clear validation error.

6 — the mutation returns a pre-transaction snapshot

`return workflow` at `routers.ts:199` returns the object read before the transaction, so its `nodes` are stale. Currently harmless because the client invalidates `getOne` and only reads `name` / `id`, but it will mislead the next caller who trusts the response.

7 — dead `Node.credentialId` column

The column is nullable, carries an `ON DELETE SET NULL` foreign key, and is not written by the editor or by the save path. Credentials are resolved inside executors instead. It is a leftover from a design in which a node referenced its own credential row.

What is genuinely solid

Ownership is enforced in the `where` clause at the one place it must be, so the unscoped `deleteMany` that follows is safe. Cascade deletes make node removal and edge cleanup consistent without extra logic. Merging rather than recreating nodes protects configuration against partial snapshots. And the dirty-flag logic is not merely defensive but precisely matched to React Flow's diff semantics — the `select` and `dimensions` exclusions are exactly the two change kinds that carry no persisted state.